Aetheris · 迭代总结报告（2026-08-25 ~ 2026-08-30）

> 承接 SUMMARY-2026-08-23.md。本文覆盖最近几天的完整迭代： 真实数仓底座（Phase 0）、分层元数据与 SQL 血缘（Phase 1）、框架升级（Spring Boot 3.4.7 + Spring AI 1.0.0）、任务编排执行引擎（Phase 2）、数据智能体 DataAgent（Phase 3）， 以及账户自助与全局搜索等系统基础能力。定位是「面向完全没看过本项目的人，也能读懂做了 什么、怎么做的、现在是什么状态、下一步往哪走」。

---

0. 一句话现状

Aetheris 已经从一个「有登录、有页面」的骨架，演进成一个**具备真实执行闭环的数据平台雏形**：

- 有**真实数仓**：WSL 上跑 Spark Thrift Server，五层库（ODS/DWD/DWS/ADS/DIM）约 130 万订单、 320 万明细、180 个分区，ODS 层刻意注入 11 类脏数据。
- 有**真实元数据**：从 Hive 反采集库表与字段（含表注释），自动推断数仓分层，从加工 SQL 解析出 17 条真实血缘边，不是种子数据。
- 有**任务编排执行**：Quartz JDBC JobStore 调度，任务按业务日期幂等，SQL 写入走白名单守卫， 依赖拓扑支持上游未就绪跳过。
- 有**数据智能体**：自然语言提问 → AI 自主检索元数据 → 生成只读 SQL → 执行 → 给出带真实 数字的回答，实测能正确回答「2026 年 8 月销售额最高的品类」这类聚合问题。
- 有**安全边界**：读写账号分离、SQL 只读/写入双守卫、内部端点不经网关暴露、JWT + 最小 RBAC。

---

1. 时间线总览

日期	里程碑	对应提交
08-23	骨架闭环、JWT 认证、AI 对话、模块铺开	`2bee21f` 及之前
08-25	数据源连通性与管理加固	`4a832eb`
08-30	Phase 0：五层数仓 + 电商域造数	`494f30e`
08-30	Phase 1：分层元数据 + 数仓地图 + SQL 血缘解析	`97434a4`
08-30	框架升级：Boot 3.4.7 + Spring AI 1.0.0	`29c598d`
08-30	Phase 2：任务编排服务 + SQL 加工执行引擎	`60583a4`
08-30	Phase 3：DataAgent 打通自然语言数据问答	`fe8bd41`
08-30	网关 CORS 修复、控制台视觉改版	`8d72f79` / `24b05de`
08-30	账户自助 + 全局搜索	`eb191be`

下面按「阶段」而非「日期」组织，因为同一天内的多笔提交其实是递进的几个阶段。

---

2. Phase 0 —— 真实数仓底座

2.1 目标

前几个迭代的元数据、血缘、AI 问答都在用**种子数据**（`ods_orders → dwd_orders` 这类表名， 实际并不存在）。这一阶段要把它替换成**真实跑在 Hive/Spark 上的数仓**，让后续的元数据采集、 血缘解析、任务执行、DataAgent 都有真实作用对象。

2.2 技术实现

- **运行载体**：WSL 内的 Spark Thrift Server（HiveServer2 兼容），元数据存 Derby，数据文件 落 HDFS（本地模式）。
- **分层建库**：`ods` / `dwd` / `dws` / `ads` / `dim` 五层库，电商域 16 张核心表。
- 事实表（订单、明细）按 `dt` 分区；维度表与 ADS 结果表不分区。
- **造数脚本**：`scripts/gen-warehouse-data.sh`，分六步可重跑（建库 / ODS / DWD / DWS / ADS / 校验）， 支持 `tiny` / `wsl` / `cluster` 三档规模。 `wsl` 档约 130 万订单、320 万明细、180 个分区。
- **脏数据是有意设计**：ODS 层保留 11 类脏数据（空值、重复行、非法枚举、负价格、格式不一致、 逻辑删除未过滤等），DWD 逐条清洗。这些是后续质量规则与 DataAgent 的真实对象。
- **业务规律**：数据不是均匀噪声——周末效应（约 1.6×）、增长趋势、金额右偏、下单时段集中， 保证「上个月」「近 7 天」这类自然语言问题始终有对应分区。

2.3 踩坑与修复

问题	根因	修复
Spark driver OOM	`local[N]` 下 driver 兼作 executor，1g 堆写百万订单即爆	默认 3g，可配 `AETHERIS_SPARK_DRIVER_MEMORY`
动态分区写耗尽堆	单个 task 同时持有全部分区的 Parquet writer	写分区语句末尾统一补 `DISTRIBUTE BY dt`
restart 竞态	`stop` 异步 SIGTERM， `start` 见端口仍监听就跳过	`stop` 等待端口释放后返回

> 详细分层约定、表清单、脏数据比例、口径定义见 docs/warehouse-layers.md。

---

3. Phase 1 —— 分层元数据 + SQL 血缘

3.1 分层元数据

`metadata_table` 新增 `layer` / `domain` / `layer_source` 三列（Flyway V7）：

- **分层推断**：采集时按库名推断层级，库名推断不出时退回表名前缀。
- 规则：层级名后必须跟分隔符（`_` / `-` / `.`）或完全相等，否则 `adsense_log` 会被误判成 `ads`、`odyssey_events` 被误判成 `ods`；匹配不上归入 `other`。
- **人工优先**：人工指定过的层级 `layer_source=manual` 不会被重复采集覆盖，与 `source`（手工/自动）的取舍逻辑一致。
- **数仓地图**：`/api/metadata/tables/warehouse-map` 按数据流向（ODS → DWD → DWS → ADS）聚合， 空层级也返回，避免前端分层图错位。

3.2 表注释采集（NL2SQL 的关键）

Hive/Spark 的 `SHOW TABLES` 与 JDBC `getTables()` 都**不返回 REMARKS**，导致表注释全空。 修复：采集时逐表执行 `DESCRIBE FORMATTED` 读取 Comment 行。这笔额外查询是值得的—— 表注释是 NL2SQL 命中率最重要的信息来源（「客单价」能否命中 `avg_order_amount`，全靠注释与指标定义）。

3.3 SQL 血缘解析

`/api/metadata/lineages/parse` 从加工 SQL 提取表级血缘。

为什么不用 JSqlParser：实测 4.9 版无法解析 Hive 的 `INSERT OVERWRITE TABLE ... PARTITION (dt)`， 本项目的四类真实写法全部抛 `JSQLParserException`。因此改用**面向 Hive 方言的提取器**，覆盖 `INSERT OVERWRITE/INTO`、`LATERAL VIEW`、`DISTRIBUTE BY`、`CREATE TABLE AS SELECT`。

排除项（否则产生不存在的表节点）：CTE 名称、表值函数（`range()` / `EXPLODE()` / `SEQUENCE()`）、 自引用、注释与字符串字面量里的 `from` / `join`。

幂等：同一目标表重复解析时先删除既有解析边再写入，SQL 改写后不残留过期依赖； 人工登记的血缘（`source=manual`）不受解析影响， `lineage_relation` 对 (上游, 下游) 建唯一索引。

当前为**表级**血缘，不追踪字段级加工路径（需要完整 AST，等引入真正支持 Hive 方言的解析器）。

---

4. 框架升级 —— Spring Boot 3.4.7 + Spring AI 1.0.0

4.1 版本选择的依据

升级前查询了 Maven Central **实际发布的构件**（而非二手资讯），确认：

- Spring AI 最新为 **1.0.0**，其 starter 自身依赖 `spring-boot-starter 3.4.5`，因此 Boot 选 3.4.7 对齐。
- Spring Boot 最新为 3.5.x，**不存在 4.x 构件**；Spring Framework 7.x 只有里程碑版（M1–M6），无 GA。
- MyBatis-Plus 无 `boot4-starter`， `boot3-starter:3.5.7` 是当前最新且兼容 Boot 3.4。

网络上「Spring AI 2.0 GA / Spring Boot 4.x 已发布」的说法与 Maven Central 事实不符，升级以能实际拉到的构件为准。

4.2 落地

- 全部 8 个后端模块升级到 Boot 3.4.7 / Spring Cloud 2024.0.1 / springdoc 2.8.6，零代码改动通过全量测试。
- `ai-agent` 引入 Spring AI 1.0.0，DeepSeek 调用改走 `spring-ai-starter-model-deepseek`，替代手写 RestClient。

- 新增 `LlmClient` 端口 + `SpringAiLlmClient` 适配器：业务逻辑不再绑定模型厂商，换模型只替换适配器。
- 模型失败按 `NOT_CONFIGURED / RATE_LIMITED / UNAVAILABLE / UNKNOWN` 分类，分别映射 503 / 429 / 502。
- 修复限流器内存泄漏（`ConcurrentHashMap` 累积过期条目）+ 并发缺陷（CAS 重置窗口在多线程下放大配额，改用 `compute` 原子创建）。

---

5. Phase 2 —— 任务编排与 SQL 加工执行引擎

5.1 为什么执行能力放在 metadata 而不是 workflow

任务执行刻意拆成两个服务：

```
workflow-service                    metadata-service
  任务定义 / Quartz 调度              数据源凭据 / JDBC 驱动
  实例状态 / 日志 / 重跑      ──→     URL 协议校验 / 参数黑名单
  依赖拓扑                            SqlWriteGuard 白名单
                                      ↓
                                  Hive / Spark / PostgreSQL
```

metadata 已持有凭据、驱动（含 hive-jdbc 的 12 处 exclusion）、`buildUrl` 的协议校验和危险连接 参数黑名单。把这些复制到 workflow 不只是重复代码，而是**复制一份安全边界**——两边任一处遗漏 就是缺口。走 `/internal/**` 路径，网关只路由 `/api/**`，该端点仅在 Compose 内部网络可达（实测外部返回 404）。

5.2 Quartz 调度

- 用 JDBC JobStore 且 `isClustered=true`：调度落库，服务重启不丢；多实例由 `QRTZ_LOCKS` 行锁保证同一次触发只被一个实例执行。
- Quartz 表结构从 2.5.0 发行包提取后脚本转换生成迁移（移除 DROP、加 IF NOT EXISTS），已比对 2.3.2（运行时版本）与 2.5.0 列定义，确认零差异。
- 定时触发的业务日期取「昨天」，misfire 策略 `DoNothing`：停机期间错过的触发不补跑，避免用错误的业务日期执行；补数据走手动触发显式指定日期。

5.3 幂等设计（两层）

- **实例幂等**：`(task_id, biz_date)` 唯一，重跑复用同一行并累加 attempt，实例列表不因反复重跑膨胀，也排除了同一业务日期被并发写入。
- **数据幂等**：依赖加工语句用 `INSERT OVERWRITE ... PARTITION` 覆盖目标分区，重跑不产生重复数据，不依赖事务回滚（Hive/Spark 本就不支持事务性 DML）。

5.4 SQL 守卫

两条 SQL 路径的守卫：

路径	守卫	放行	用途
只读查询	`validateReadOnlySql`	`SELECT` / `SHOW` / `DESCRIBE` / `EXPLAIN`	数据源浏览、AI 助手
写入执行	`SqlWriteGuard`	`INSERT INTO/OVERWRITE`、`CREATE ... AS SELECT`	数仓加工任务

两条路径都拒绝多语句、`DROP` / `TRUNCATE` / `ALTER` / `GRANT` 与可读服务器文件的函数。 写入白名单刻意不含 `DROP`：`INSERT OVERWRITE` 覆盖分区已能满足重跑，不需要先删表。

守卫顺序缺陷的修复（由测试发现）：白名单与分号检查原先用原始文本，导致以注释开头的真实 加工脚本被误拒、注释里的分号被误判为多语句。改为**先剥离注释与字符串字面量再检查**。

动态分区参数（`hive.exec.dynamic.partition`）由服务端注入而非让用户写在 SQL 里——守卫只放行单条语句，用户无法自带 `SET`。

5.5 依赖拓扑与占位符

- 依赖未就绪标记 `skipped` 而非 `failed`，避免告警噪声掩盖真正的故障；登记依赖时检测环形与自依赖并拒绝。
- 任务 SQL 支持 `${bizDate}` 占位符，同一份定义可跑任意业务日期，补数据无需改任务。
- 任务保存时即校验 SQL 白名单（走 metadata 的 `/internal/sql/validate-write`），修复「保存成功、点运行才失败」的问题；校验与执行共用同一份 `SqlWriteGuard`，不在 workflow 复制规则以免漂移。

5.6 端到端验收

任务写入 Hive 的 `order_cnt=5067 / gmv=6102169.27 / user_cnt=5067` 与独立计算完全一致，且与 Phase 0 的 `ads_order_stats` 交叉印证。

---

6. Phase 3 —— DataAgent（NL2SQL 数据问答）

6.1 完整链路

```
用户提问（数据模式）
      ↓
ChatController          协议转换、限流
      ↓
ChatService             会话编排、持久化
      ↓
SpringAiLlmClient       ChatClient 驱动工具调用循环
      ↓  ↑
DataTools               @Tool 注解暴露给模型的三个工具
      ↓  ↑
AgentToolClient  ──→    metadata /internal/agent/**
                          searchTables / describeTable / query
```

```
                    ↓
Hive / Spark（只读账号）
```

工具调用循环由 Spring AI 的 `ChatClient` 内建驱动（模型请求调用 → 框架执行 → 回填结果 → 再次询问模型），不需要自己实现多轮循环。

6.2 三个工具的职责与约束

工具	作用	约束
`searchTables`	按关键词与层级检索表	排除 stale 表，按层级排序让 ADS/DWS 优先
`describeTable`	读取字段名、类型、**注释**	必须先检索拿到 id
`query`	执行只读 SQL	走 `validateReadOnlySql`，最多 100 行

工具失败时返回带 `error` 字段的结构化结果而非抛异常——模型看到原因后能自行改写重试， 抛异常会中断整轮对话反而拿不到结果。检索无结果时附带改写建议，避免模型凭空编造表名。

6.3 系统提示固化的四条纪律

都对应本项目数仓的真实特征：

- **必须先检索再看结构再查询** —— 避免凭字段名猜测拼 SQL；
- **统计优先用 ADS/DWS** —— ODS 层保留脏数据，直接统计得错误结论；
- **查询必须带 dt 分区条件** —— 事实表百万级且按 dt 分区，不带条件会全表扫描超时；
- **以字段注释为口径依据** —— 「有效订单」「GMV」在本项目有固定定义，不让模型自行发明。

实测效果：问「2026 年 8 月销售额最高的品类」，agent 自主选择了 ADS 层、带上了 `month_key` 过滤， 回答正确（家用电器 / 43,094,480.70 / 占比 16.19% / 排名第 1），并说明来源表与口径。

6.4 三层防护

1. **语句守卫**： `validateReadOnlySql` 只放行 SELECT/SHOW/DESCRIBE/EXPLAIN，agent 生成的 SQL 与人工输入受同一套约束，未放宽任何限制；
2. **账号隔离**：agent 使用只读账号，即便守卫被绕过也无写权限；
3. **网络隔离**：工具端点不经网关暴露（ `/internal/**` ）。

6.5 元数据检索：暂用关键词而非向量

当前 `metadata_table` 约 20 张表，全部表名与注释可直接进入上下文，关键词检索（匹配表名、表注释、 业务域）已足够。pgvector 的价值在表数达数百张时才显现，且切换镜像需从 Alpine(musl) 跨到 Debian(glibc)，collation 行为差异会影响文本索引。待表数量增长后再引入。

6.6 读写账号分离

数仓加工任务用 HDFS 仓库属主身份连接（写权限），只读查询与 AI 助手用 `aetheris` 账号 （仓库目录对其他用户为 `r-x` ）。这不是权宜之计：DataAgent 需要「即使 SQL 守卫被绕过也无法写入」 的兜底保证，因此两个账号不合并。

---

7. 系统基础能力 —— 账户自助 + 全局搜索

> 本轮收尾工作，补齐「账户设置、搜索等系统基础功能不完善」的短板，企业级地做。

7.1 账户自助（`/api/auth/me`）

- 新增 `AuthProfileController`，身份一律取自网关注入的 `X-User-Name`（网关校验 JWT 后写入），**不接受请求体或路径参数指定用户**，否则任何登录用户都能改别人的密码。
- `GET /api/auth/me` 返回资料（响应体不含密码哈希）；`PUT` 修改昵称/邮箱/手机号；`PUT /password` 改密。
- **防自我提权**：`UpdateProfileRequest` 刻意不含 `role` / `status` / `username` 字段，角色与状态只能走管理端（要求 admin）。与 `AuthUserAdminController` 分开的原因：管理端可改任意 用户角色，自助端只能改自己，合并在一处容易在后续迭代中漏掉「不能改自己角色」的判断。
- **改密语义**：校验当前密码（持有有效令牌 ≠ 知道密码，令牌可能被盗用或来自未登出的公共设备）、 拒绝新密码与当前相同；成功后返回 `reloginRequired: true`（JWT 无状态，旧令牌在过期前仍有效）。
- 用 `LambdaUpdateWrapper` 显式置空，允许用户清空昵称/邮箱/手机号（`updateById` 会跳过 null）。
- 测试：新增 9 个单测（auth 服务合计 18 个）。

7.2 全局搜索（导航 + 元数据表）

- 后端：`GET /api/metadata/tables/search`，关键词匹配表名/表注释/业务域，排除 stale 表，空关键词返回空列表，`LIMIT` 上限。
- 前端：`GlobalSearch` 命令面板，⌘K / Ctrl+K 唤起，导航入口本地即时匹配 + 元数据表走后端（防抖 250ms）， 合并结果，键盘上下选择、回车跳转；点表结果跳转元数据目录并预填表名筛选。
- 顶部栏清理：移除无多环境概念的 environment Select、无通知概念的铃铛按钮，搜索框从纯装饰换成可用组件，账户设置菜单项接入 `/account`。

---

8. 安全设计总览

维度	实现
认证	JWT（`X-User-Name` 由网关注入，网关覆盖客户端伪造的头）
授权	最小 RBAC：JWT 携带 `admin` / `viewer`，网关负责写操作授权
口令	BCrypt（strength 10），最小 12 位，登录失败 5 次锁 15 分钟
SQL 只读	`validateReadOnlySql`：SELECT/SHOW/DESCRIBE/EXPLAIN + 危险函数黑名单
SQL 写入	`SqlWriteGuard`：INSERT OVERWRITE/INTO、CREATE AS SELECT 白名单
账号	读写分离：加工任务用仓库属主，只读查询/AI 用 `aetheris`
网络	内部端点 `/internal/**` 不经网关暴露
错误泄露	不回显驱动/提供商原始报文

9. 当前系统状态

9.1 服务拓扑（Docker Compose）

服务	端口	职责	迁移版本
web（nginx）	5173	React 控制台	—
gateway	8080	统一入口、JWT 鉴权、路由、CORS	—
auth	8087	认证、账号、自助资料	V1–V2
system	8081	系统配置、审计日志	V1–V3
metadata	8082	数据源、元数据、血缘、只读/写入执行	V1–V8
workflow	8088	任务编排、Quartz 调度、执行引擎	V1–V2
ai-agent	8086	DeepSeek 对话、DataAgent 工具调用	V1–V2
quality	8083	质量规则（管理，执行引擎未完成）	V1
ingestion	8084	接入任务（管理）	V1
governance	8085	治理策略（管理）	V1
postgres	5433	数据持久层（每服务独立 schema）	—

`serving` 目录尚为空占位。

9.2 数据规模

- 数仓：五层 16 张核心表，约 130 万订单、320 万明细、180 个分区，ODS 层 11 类脏数据。
- 元数据：约 20 张表，带分层标签、表注释、业务域；17 条真实血缘边（SQL 解析）。

9.3 测试覆盖

- `ai-agent`：20 个（controller 9 + LLM 适配器 7 + 限流器 4）。
- `auth`：18 个（管理员 9 + 自助资料 9）。
- `workflow`：任务执行幂等、并发拒绝、上游跳过/成功、重试、占位符替换等单测。
- 前端：`tsc --noEmit` 通过。

10. 架构设计与关键决策

1. **Monorepo**：前期个人开发统一管理；后期可按领域拆多仓。顶层只保留「物理分层」，业务域下沉 `services/`。
2. **Java 微服务**：Spring Boot 3.4.7 + Spring Cloud 2024.0.1，每服务独立构建/部署，经网关统一对外。

3. **执行能力下沉 metadata**：凭据、JDBC 驱动、SQL 守卫只维护一份，workflow / ai-agent 通过 `/internal/**` 复用，不复制安全边界。
4. **数据库隔离**：每服务独立 schema，Flyway 按各自 schema 管理迁移，互不干扰。
5. **注册中心延后**：Nacos 刻意延后，避免执行引擎未稳定时扩大运维复杂度，计划 Phase 3 单独锁定版本后接入。
6. **向量检索延后**：表数少时关键词够用，避免 Alpine→Debian 的 collation 迁移成本。

---

11. 后续安排

近期（继续打磨数据智能体）

1. **语义层（指标定义）**：把「有效订单 / GMV / 客单价 / 退款率」等口径固化成结构化指标， 让 DataAgent 从「检索表 + 写 SQL」升级为「直接引用指标」，是 NL2SQL 准确率与一致性的下一个杠杆。 口径的唯一来源已在 docs/warehouse-layers.md 定好。
2. **评测集（推荐先于语义层）**：固化一批「问题 → 正确 SQL / 答案」的样例，量化每次改动对 DataAgent 准确率的影响，否则改提示词、加工具都只能靠肉眼判断。
3. **质量执行引擎**：quality 服务目前只有规则管理，缺「定时校验 + 异常告警 + 阻断下游」的执行链路； 可复用 Phase 0 的 `06_validate.sql` 作为规则基准。

中期（补齐执行引擎 + 平台化）

4. **接入（ingestion）/ 治理（governance）执行引擎**：当前同为「管理能力」，执行未完成。
5. **字段级血缘**：引入真正支持 Hive 方言的 AST 解析器，追踪字段级加工路径。
6. **向量检索**：表数增长到数百张后引入 pgvector，替换关键词检索。
7. **注册/配置中心**：Nacos，锁定 Spring Cloud Alibaba 版本。

长期（生产化）

8. **Python 数据/AI 服务**：通过 `packages/contracts` 的 OpenAPI/Protobuf 契约互联。
9. **Kubernetes + 可观测性**：日志、指标、链路追踪与完整数据治理能力。

遗留小项

- **语义色被冲成深蓝**：主题里 `colorSuccess / colorWarning / colorError` 全被设成 `#071a3d`， 导致成功/警告/错误在视觉上无法区分。企业级语义色建议恢复为独立色（绿/橙/红），只保留主色深蓝。
- `system.sys_user` 旧版资料目录暂时保留，确认无外部调用后再通过独立迁移退役。

---

12. Git 提交历史（08-25 之后）

```
eb191be feat(system): 账户自助与全局搜索，补全系统基础能力
24b05de style(web): 控制台视觉改版为深蓝主色的浅色布局
8d72f79 fix(gateway): 补齐 CORS 配置，修复浏览器登录被拒为 403
```

```
fe8bd41 feat(ai-agent): DataAgent 打通自然语言数据问答
60583a4 feat(workflow): 新增任务编排服务与 SQL 加工执行引擎
29c598d feat(ai-agent): 升级到 Boot 3.4.7 并引入 Spring AI 1.0.0
97434a4 feat(metadata): 数仓分层元数据、数仓地图与 SQL 血缘解析
494f30e feat(warehouse): 建立五层数仓与电商域测试数据
4a832eb feat: improve datasource connectivity and administration
```

---

13. 常用命令速查

```
docker compose up --build -d            # 一键构建并启动
make help                               # 查看命令
cd apps/web && npm run dev              # 前端开发模式（热更新）

# 数仓（WSL Spark Thrift Server）
bash scripts/wsl-hive-test-server.sh start   # 启动 Hive 栈
bash scripts/gen-warehouse-data.sh            # 建库 + 造数 + 逐层加工 + 校验
bash scripts/gen-warehouse-data.sh --steps 3,4,5   # 只重跑指定步骤

# WSL 动态 IP → Docker 的端口转发（采集元数据前必须）
pwsh ./scripts/windows-wsl-hive-relay.ps1 start
```